

Tier 4: Real-Time Evaluation of the Landmark Supermodel

PhysioNet Utility Score, Alarm-Burden Analysis, and Dynamic Calibration

Abdul Ahad. Student ID: G00486649

09 July 2026

Table of Contents

0.1	Overview	1
0.2	Load Libraries and Set Seed	1
0.3	Locate and Read Data	1
0.4	Design Matrix Builder	2
0.5	Train the Supermodel	2
0.6	Score Every Holdout Hour	3
0.7	PhysioNet/CinC-2019 Utility Score	3
0.8	Alarm-Burden Sweep	4
0.9	Alarm Trade-Off Figure	5

0.1 Overview

This script evaluates the landmark supermodel as a streaming early-warning system on leakage-safe holdout stays. It computes: (1) the PhysioNet/CinC-2019 utility score on an hourly risk stream, (2) an alarm-burden sweep trading off sensitivity against false alarms per patient-day, and (3) a visual summary of the alarm trade-off.

0.2 Load Libraries and Set Seed

We load `glmnet` for the elastic-net supermodel and `pROC` for discrimination metrics, then fix the random seed.

```
suppressPackageStartupMessages({ library(glmnet); library(pROC) })
set.seed(486649)
```

0.3 Locate and Read Data

A helper function searches several candidate paths for each input file. We load the landmark stack (training data), the holdout stay IDs, and the pre-computed hourly features for the holdout stays.

```
find_data <- function(f) {
  cands <- c(file.path("../..//processed_data", f),
             file.path("processed_data", f), f)
  hit <- cands[file.exists(cands)][1]; if (is.na(hit)) stop("cannot find ", f); hit
}
OUT <- dirname(find_data("landmark_stack.csv"))
```

```

FIG <- if (dir.exists("figure")) "figure" else { dir.create("figure"); "figure" }

stack <- read.csv(find_data("landmark_stack.csv"))
hold_ids <- read.csv(find_data("realtime_holdout_ids.csv"))$stay_id
hnew <- read.csv(find_data("realtime_holdout_features.csv"))

vitals <- c("hr", "resp_rate", "map", "spo2", "temp_c", "gcs")
labs <- c("lactate", "creatinine", "wbc", "pf_ratio")
slopes <- c("hr_slope6", "resp_rate_slope6", "map_slope6", "spo2_slope6")
scores <- c("sofa_total", "qsofa", "sirs")

```

0.4 Design Matrix Builder

The `make_ref` function extracts training-set medians and scaling parameters so that no holdout information leaks into imputation or standardization. The `make_design` function then builds the full feature matrix, including missingness indicators, landmark-time interactions, and scaled terms.

```

# ---- design builder: `ref` carries TRAIN medians + s mean/sd (no leakage) ----
make_ref <- function(raw) {
  med <- sapply(c(vitals, scores, labs), function(v) median(raw[[v]], na.rm=TRUE))
  list(med = med, s_mu = mean(raw$landmark_time), s_sd = sd(raw$landmark_time))
}
make_design <- function(raw, ref) {
  df <- data.frame(row.names = seq_len(nrow(raw)))
  for (v in c(vitals, scores)) { x <- raw[[v]]; x[is.na(x)] <- ref$med[[v]]; df[[v]] <- x }
  for (v in labs) { x <- raw[[v]]; df[[paste0(v, "_msg")]] <- as.integer(is.na(x));
    x[is.na(x)] <- ref$med[[v]]; df[[v]] <- x }
  for (v in slopes) { x <- raw[[v]]; df[[paste0(v, "_msg")]] <- as.integer(is.na(x));
    x[is.na(x)] <- 0; df[[v]] <- x }
  df$age <- raw$age; df$female <- as.integer(raw$gender == "F")
  base <- names(df)
  sz <- (raw$landmark_time - ref$s_mu) / ref$s_sd
  df$s <- sz; df$s2 <- sz^2
  for (v in base) df[[paste0(v, "_xS")]] <- df[[v]] * sz
  as.matrix(df)
}

```

0.5 Train the Supermodel

We exclude the holdout stays from the landmark stack and fit a cross-validated elastic-net (alpha = 0.5) logistic regression. This is the same supermodel architecture used elsewhere in the pipeline.

```

# ---- train supermodel on stack minus holdout stays -----
tr <- stack[!(stack$stay_id %in% hold_ids), ]
cat(sprintf("train rows %d (%d stays); holdout %d stays, %d scored hours\n",
  nrow(tr), length(unique(tr$stay_id)), length(hold_ids), nrow(hnew)))

```

```
## train rows 188119 (44389 stays); holdout 6000 stays, 152910 scored hours
```

```
ref <- make_ref(tr)
Xtr <- make_design(tr, ref); ytr <- tr$onset_within_h
cvfit <- cv.glmnet(Xtr, ytr, family="binomial", alpha=0.5, nfolds=5, standardize=TRUE)
```

0.6 Score Every Holdout Hour

We apply the trained supermodel to every hour of the holdout stays, producing a continuous risk stream that subsequent sections evaluate.

```
# ---- score every holdout hour -> hourly risk stream -----
Xho <- make_design(hnew, ref)
hnew$risk <- as.numeric(predict(cvfit, Xho, s="lambda.min", type="response"))
```

0.7 PhysioNet/CinC-2019 Utility Score

We compute the normalized utility score across a grid of alarm thresholds. The reward window places optimal value at 6 hours before onset, with linearly decaying rewards earlier and later. The normalized score is 0 for a “never-alarm” policy and 1 for a per-hour oracle.

```
# ---- (1) PhysioNet/CinC-2019 utility score -----
# reward window relative to onset t*: early -12h, optimal -6h, late +3h.
DT_EARLY <- -12; DT_OPT <- -6; DT_LATE <- 3
U_FP <- -0.05; MIN_FN <- -2; MAX_TP <- 1
# per-hour utility given predicted 0/1 and relative time tau = hour - onset
u_pred1 <- function(tau, septic) {
  ifelse(!septic, U_FP,
    ifelse(tau <= DT_EARLY, U_FP,
      ifelse(tau <= DT_OPT, (tau - DT_EARLY)/(DT_OPT - DT_EARLY) * MAX_TP,
        ifelse(tau <= DT_LATE, MAX_TP*(1 - (tau - DT_OPT)/(DT_LATE - DT_OPT)), 0))))
}
u_pred0 <- function(tau, septic) {
  ifelse(!septic, 0,
    ifelse(tau <= DT_OPT, 0,
      ifelse(tau <= DT_LATE, MIN_FN*(tau - DT_OPT)/(DT_LATE - DT_OPT), MIN_FN)))
}
septic <- !is.na(hnew$onset_hour)
tau <- hnew$hour - ifelse(septic, hnew$onset_hour, Inf)

utility_at <- function(thresh) {
  pred <- hnew$risk >= thresh
  u_model <- sum(ifelse(pred, u_pred1(tau, septic), u_pred0(tau, septic)))
  u_model
}
u_no <- sum(u_pred0(tau, septic)) # never alarm
u_best <- sum(pmax(u_pred1(tau, septic), u_pred0(tau, septic))) # oracle per-hour
grid <- seq(0.02, 0.6, by = 0.01)
u_raw <- sapply(grid, utility_at)
norm <- (u_raw - u_no) / (u_best - u_no)
best_i <- which.max(norm)
```

```

util_tab <- data.frame(threshold = grid,
                      utility_raw = round(u_raw, 1),
                      utility_norm = round(norm, 4))
write.csv(util_tab, file.path(OUT, "tier4_utility.csv"), row.names=FALSE)
cat(sprintf("\nPhysioNet utility (normalized): best %.4f at threshold %.2f\n",
           norm[best_i], grid[best_i]))

##
## PhysioNet utility (normalized): best 0.2886 at threshold 0.04
cat(sprintf(" (never-alarm baseline utility_norm = 0; per-hour oracle = 1)\n"))

## (never-alarm baseline utility_norm = 0; per-hour oracle = 1)

```

0.8 Alarm-Burden Sweep

For each threshold we compute patient-level sensitivity (fraction of septic patients who receive a timely alarm) and the false-alarm rate per patient-day. This lets clinicians choose an operating point that balances early detection against alarm fatigue.

```

# ---- (2) alarm-burden sweep: patient-level sensitivity vs false alarms ----
# an alarm "counts" for a septic patient if it fires at/after t_early and at or
# before onset+late (a timely catch); false alarms = alarm-hours on non-septic
# patients (or septic hours outside the reward window).
sweep <- lapply(grid, function(th) {
  pred <- hnew$risk >= th
  # patient-level: did we catch each septic patient in a timely way?
  timely <- pred & septic & (tau >= DT_EARLY) & (tau <= DT_LATE)
  caught <- tapply(timely, hnew$stay_id, any)
  is_sep_pt <- tapply(septic, hnew$stay_id, any)
  sens <- mean(caught[is_sep_pt], na.rm=TRUE)
  # false-alarm hours: alarm on a non-septic patient, or septic before window
  fa <- pred & ((!septic) | (septic & tau < DT_EARLY))
  fa_per_day <- sum(fa) / sum(!septic | (septic & tau < DT_EARLY)) * 24
  # non-septic patients with >=1 false alarm (alarm-fatigue proxy)
  nonsep_pt <- tapply(!septic, hnew$stay_id, all)
  any_fa <- tapply(pred & !septic, hnew$stay_id, any)
  fpr_pt <- mean(any_fa[nonsep_pt], na.rm=TRUE)
  data.frame(threshold=th, sensitivity=sens, fa_per_patient_day=fa_per_day,
             frac_nonseptic_pts_alarmed=fpr_pt)
})
sweep <- do.call(rbind, sweep)
sweep[, -1] <- round(sweep[, -1], 4)
write.csv(sweep, file.path(OUT, "tier4_alarm_sweep.csv"), row.names=FALSE)
# report a couple of operating points
for (target in c(0.90, 0.70, 0.50)) {
  i <- which.min(abs(sweep$sensitivity - target))
  cat(sprintf(" sens=%.2f @ thr %.2f -> %.2f false alarms/patient-day, %.0f%% of non-septic pat
             sweep$sensitivity[i], sweep$threshold[i],
             sweep$fa_per_patient_day[i], 100*sweep$frac_nonseptic_pts_alarmed[i]))
}

```

```
}
```

```
## sens=0.90 @ thr 0.02 -> 6.51 false alarms/patient-day, 86% of non-septic patients ever alar
## sens=0.72 @ thr 0.05 -> 2.00 false alarms/patient-day, 59% of non-septic patients ever alar
## sens=0.51 @ thr 0.09 -> 0.70 false alarms/patient-day, 30% of non-septic patients ever alar
```

0.9 Alarm Trade-Off Figure

This plot visualizes the sensitivity-versus-false-alarm trade-off, highlighting operating points near 90% and 70% sensitivity.

```
# ---- figure: alarm trade-off -----
png(file.path(FIG, "alarm_tradeoff.png"), width=1100, height=850, res=200)
plot(sweep$fa_per_patient_day, sweep$sensitivity, type="l", col="firebrick", lwd=2,
     xlab="False alarms per patient-day", ylab="Timely detection sensitivity",
     main="Alarm burden of the real-time supermodel")
pts <- c(which.min(abs(sweep$sensitivity-0.9)), which.min(abs(sweep$sensitivity-0.7)))
points(sweep$fa_per_patient_day[pts], sweep$sensitivity[pts], pch=19)
text(sweep$fa_per_patient_day[pts], sweep$sensitivity[pts],
     sprintf("thr=%.2f", sweep$threshold[pts]), pos=4, cex=0.7)
dev.off()
```

```
## pdf
```

```
## 2
```

```
cat("\nTier 4 complete.\n")
```

```
##
```

```
## Tier 4 complete.
```